

ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

UNIT 4
KIẾN TRÚC ỨNG DỤNG VÀ DI CHUYỂN GIỮA CÁC MÀN HÌNH

ONE LOVE. ONE FUTURE.

Mục đích

- Giải thích vòng đời Activity
- Tổ chức UI state bằng ViewModel
- Xây luồng điều hướng nhiều màn hình
- Thiết kế giao diện thích ứng

Nội dung

- Các thành phần trong kiến trúc của ứng dụng
- Di chuyển giữa nhiều màn hình
- Giao diện thích ứng

HUST

hust.edu.vn fb.com/dhbkhn

PHẦN 1 – Architecture components

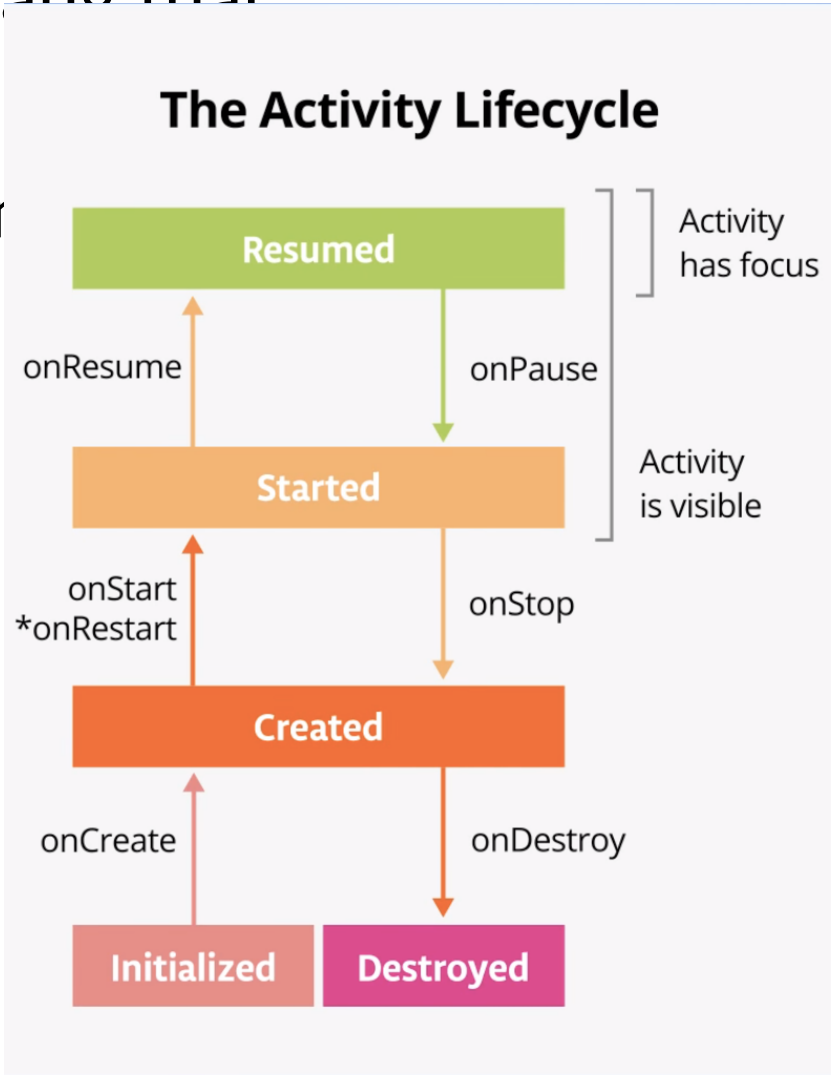
- **Vòng đời của activity**
- Kiến trúc ứng dụng
- Lớp giao diện
- ViewModel và State

Ứng dụng lớn cần kiến trúc rõ ràng

- Nhiều màn hình làm tăng độ phức tạp
- Trạng thái phải nhất quán
- Logic cần tách khỏi giao diện
- Mã nguồn phải dễ kiểm thử

Lifecycle là máy trạng thái của Activity

- Activity chuyển qua nhiều trạng thái
- Hệ thống gọi callback tương
- Chuyển trạng thái có thể tiến
- Mỗi callback có trách nhiệm



Các trạng thái khi ứng dụng xuất hiện

onCreate() → onStart() → onResume()

Trạng thái	Ý nghĩa
CREATED	Khởi tạo một lần cho instance
STSTARTED	Activity đã nhìn thấy
RESUMED	Activity ở foreground, có focus và tương tác được

Các trạng thái khi activity rời màn hình

onPause() → onStop() → onDestroy()

Trạng thái	Ý nghĩa
PAUSED	Activity bị che một phần, mất focus, không tương tác được
STOPPED	Activity bị che hoàn toàn
DESTROYED	Giải phóng acitivity

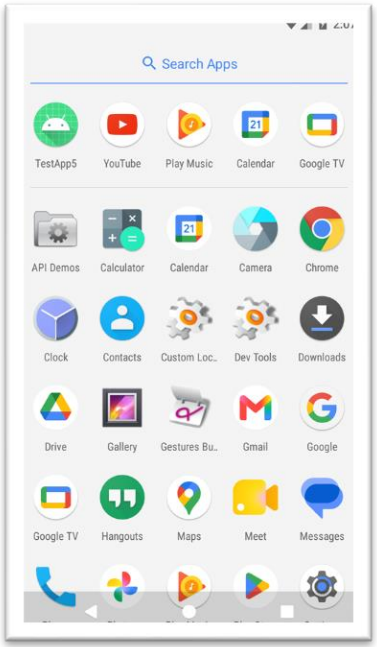
Ví dụ minh họa quát sát vòng đời activity

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    enableEdgeToEdge()
    setContent {
        Greeting("Android")
    }
    Log.v("TAG", "onCreate")
}

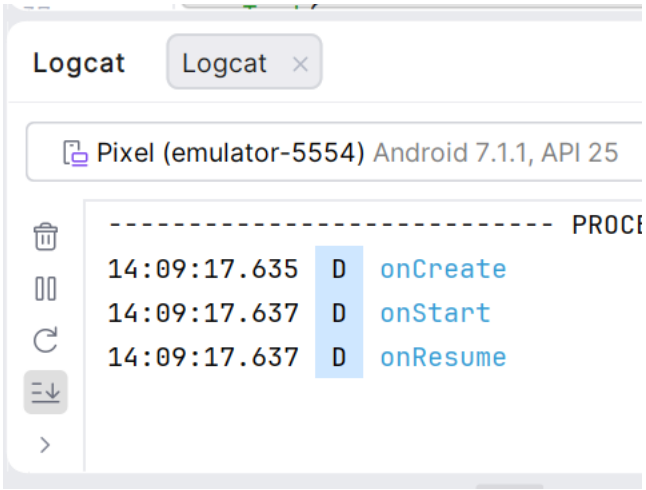
override fun onStart() { super.onStart(); Log.d("TAG", "onStart") }
override fun onResume() { super.onResume(); Log.d("TAG", "onResume") }
override fun onPause() { Log.d("TAG", "onPause"); super.onPause() }
override fun onStop() { Log.d("TAG", "onStop"); super.onStop() }
override fun onRestart() { super.onRestart(); Log.d("TAG", "onRestart") }
override fun onDestroy() { Log.d("TAG", "onDestroy"); super.onDestroy() }
```

Trình tự khởi động Activity

- onCreate(): khởi tạo đối tượng
- onStart(): Activity bắt đầu hiển thị
- onResume(): người dùng có thể tương tác
- Chuỗi gọi: Create → Start → Resume

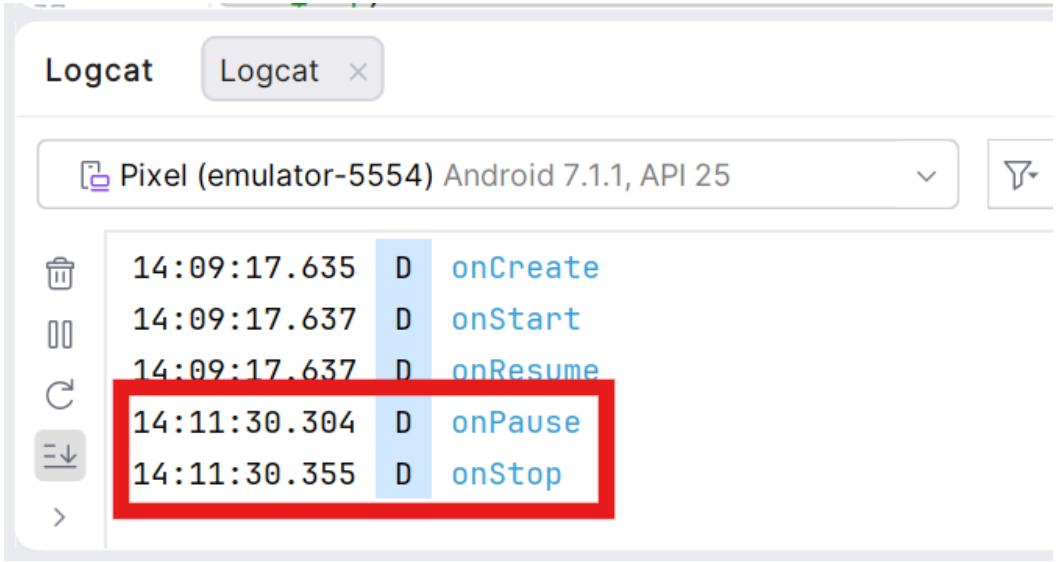






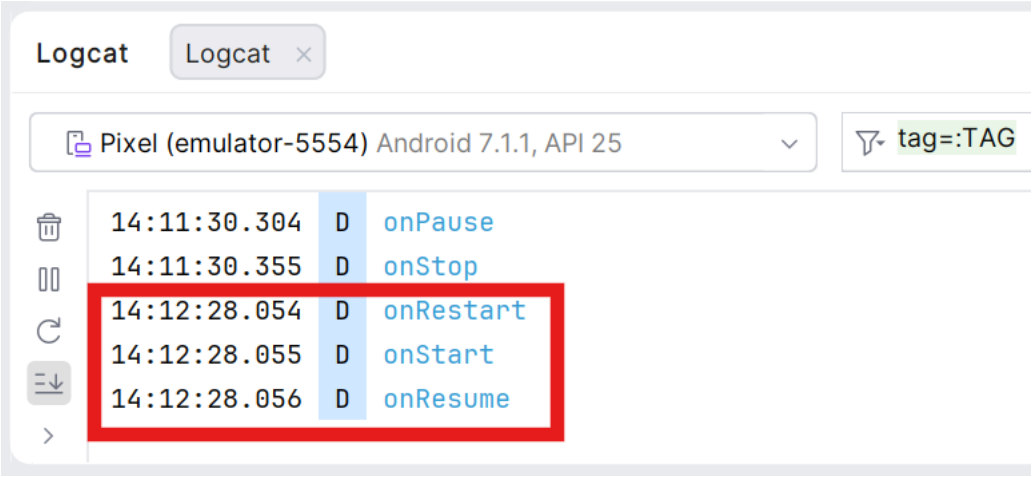
Nhấn Home đưa Activity về nền

- onPause() → onStop()
- Activity không còn hiển thị và không nhận tương tác
- Instance thường vẫn ở trong bộ nhớ
- Không giả định tiến trình luôn tồn tại (có thể bị giải phóng bởi hệ điều hành)



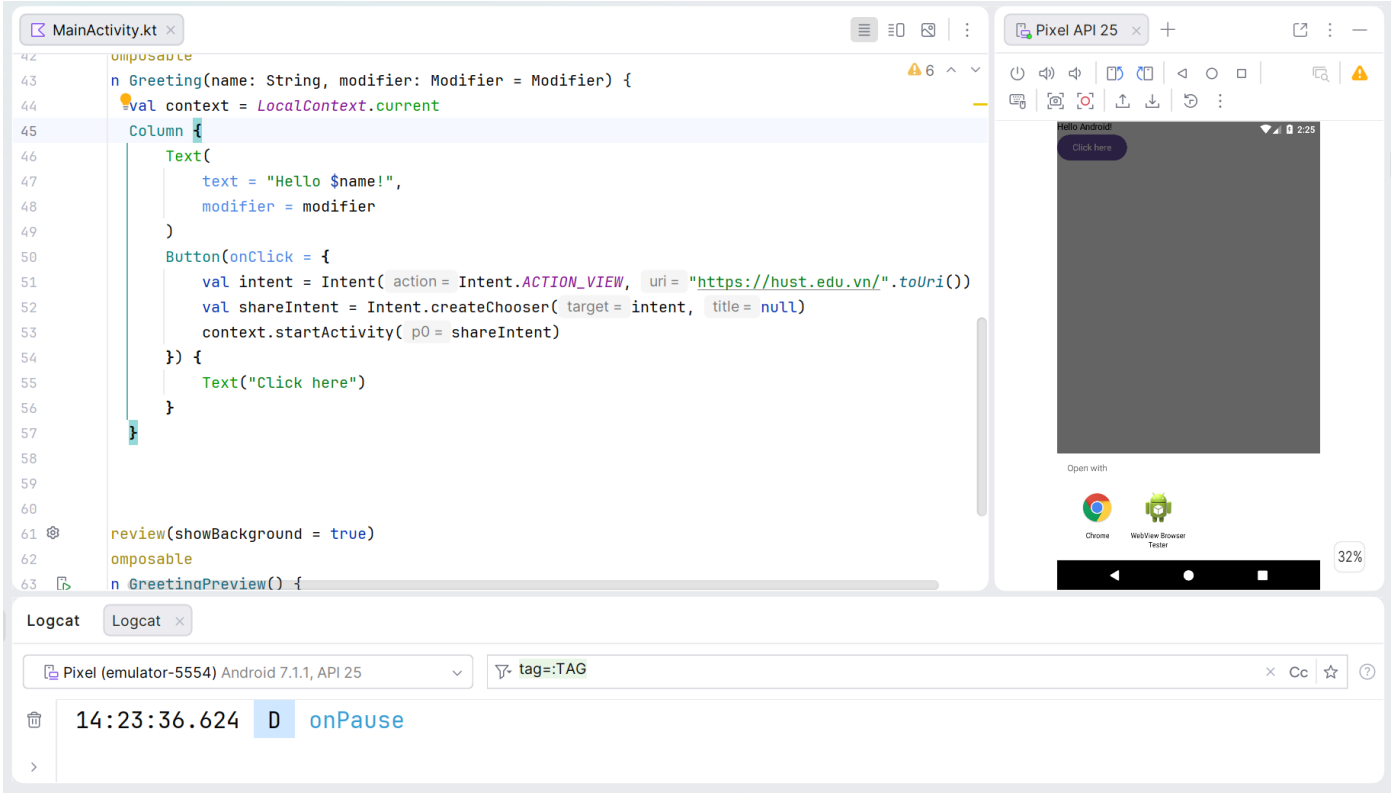
Quay lại ứng dụng sau khi nhấn Home

- onRestart() → onStart() → onResume()
- Activity hiển thị và khôi phục tương tác
- Dữ liệu trong instance có thể còn tồn tại



Activity có thể còn thấy nhưng mất focus

- Activity bị che một phần (bởi Dialog)
- Thông thường chỉ onPause() được gọi
- UI vẫn có thể tiếp tục thực hiện cập nhật



Back và Home tạo kết quả khác nhau

Nhấn HOME

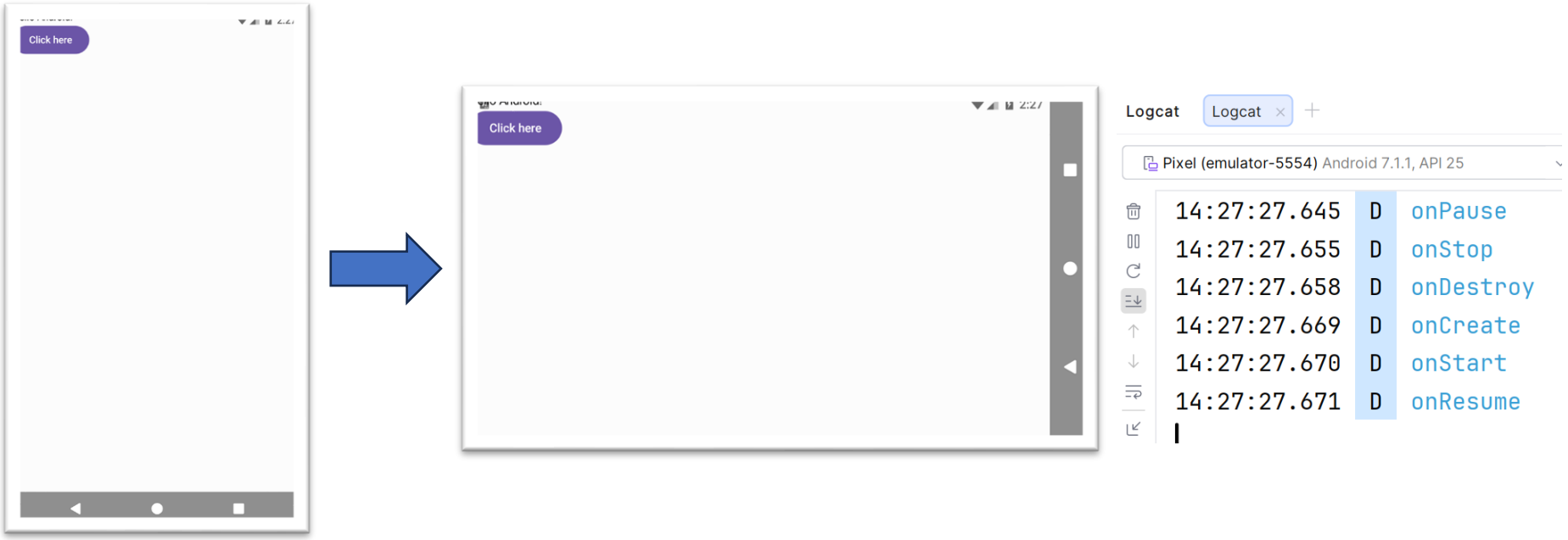
- Ứng dụng vào nền
- onPause() → onStop()
- Có thể quay lại instance cũ

Nhấn BACK

- Thoát ứng dụng
- onPause() → onStop() → onDestroy()
- Acitivity bị giải phóng nên không quay lại được instance cũ

Configuration change tái tạo Activity

- Khi xoay màn hình hoặc thay đổi ngôn ngữ
- Activity cũ bị dừng và hủy
- Activity mới được tạo lại
- UI phải khôi phục trạng thái cần thiết



Chuỗi callback khi xoay màn hình

- Instance cũ: Pause → Stop → Destroy
- Hệ thống áp dụng cấu hình mới
- Instance mới: Create → Start → Resume
- *Không coi rotation là lỗi*

Đặt công việc đúng callback

onCreate

- Khởi tạo UI một lần
- Thiết lập dependency
- Gọi setContent()

onStart / onStop

- Bắt đầu / dừng tài nguyên
- Theo khả năng hiển thị
- Bảo toàn pin và CPU

Không làm việc nặng trong onPause

- Callback chặn chuyển tiếp giao diện
- Ghi dữ liệu nhỏ nếu thật sự cần
- Đẩy tác vụ dài sang tầng phù hợp
- Ưu tiên phản hồi hệ thống nhanh

Giới hạn của saved instance state

- Bundle có giới hạn kích thước
- Chỉ lưu dữ liệu tối thiểu để khôi phục
- Dữ liệu nghiệp vụ nên ở data layer
- Tải lại dữ liệu khi cần

Phân loại trạng thái trước khi lưu

UI ELEMENT STATE

- Nội dung đang nhập
- Vị trí cuộn
- Lựa chọn tạm thời

SCREEN UI STATE

- Dữ liệu màn hình
- Kết quả nghiệp vụ
- Do ViewModel quản lý

remember giữ state qua recomposition

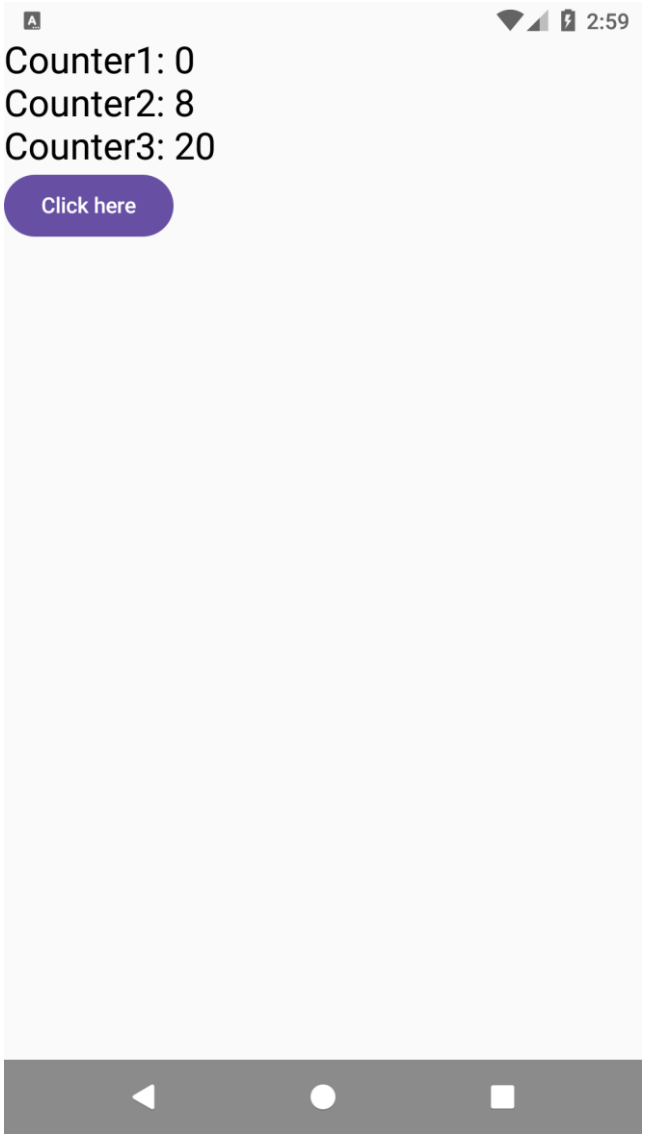
- Giá trị tồn tại trong Composition
- Không bị tạo lại mỗi recomposition
- Mất khi composable rời Composition
- Khởi tạo lại khi Activity bị tái tạo

rememberSaveable và tái tạo UI

- Lưu kiểu có thể ghi vào Bundle
- Khôi phục sau configuration change
- Phù hợp cho UI element state nhỏ
- Không dùng cho dữ liệu lớn

Ví dụ minh họa

```
@Composable
fun Counting(modifier: Modifier = Modifier) {
    var counter1: Int = 0
    var counter2 by remember { mutableIntStateOf(0) }
    var counter3 by rememberSaveable { mutableIntStateOf(0) }
    Column(Modifier.fillMaxSize().systemBarsPadding()) {
        Text(text = "Counter1: $counter1", fontSize = 24.sp)
        Text(text = "Counter2: $counter2", fontSize = 24.sp)
        Text(text = "Counter3: $counter3", fontSize = 24.sp)
        Button(onClick = {
            counter1++
            counter2++
            counter3++
        }) {
            Text("Click here")
        }
    }
}
```



Codelab

https://developer.android.com/codelabs/basic-android-kotlin-compose-activity-lifecycle#0

HUST

hust.edu.vn fb.com/dhbkhn

PHẦN 1 – Architecture components

- Vòng đời của activity
- **Kiến trúc ứng dụng**
- Lớp giao diện
- ViewModel và State

Khái niệm

Kiến trúc ứng dụng Android là cách tổ chức các thành phần, dữ liệu và trách nhiệm trong ứng dụng để chúng phối hợp với nhau rõ ràng.

Kiến trúc tốt giúp ứng dụng:

- Dễ phát triển và bảo trì.
- Dễ kiểm thử.
- Hạn chế lỗi khi xoay màn hình hoặc thay đổi cấu hình.
- Dễ mở rộng khi ứng dụng có thêm chức năng.

Các nguyên tắc quan trọng

- Separation of concerns
- Adaptive layouts
- Drive UI from data models
- Single source of truth
- Unidirectional data flow

Separation of concerns

Mỗi thành phần chỉ nên đảm nhiệm một nhóm trách nhiệm rõ ràng:

- Composable: hiển thị giao diện.
- ViewModel: quản lý trạng thái giao diện.
- Repository: quản lý và cung cấp dữ liệu.
- Data source: làm việc trực tiếp với nguồn dữ liệu.

Không nên viết toàn bộ giao diện, truy vấn Room, gọi API và xử lý nghiệp vụ trong một Activity.

Adaptive layouts

Giao diện phải tự điều chỉnh theo không gian hiển thị hiện có, thay vì giả định ứng dụng luôn chạy trên điện thoại ở chế độ dọc.

Ứng dụng Android có thể chạy trên:

- Điện thoại.
- Máy tính bảng.
- Thiết bị gập.
- Chromebook.
- Cửa sổ có thể thay đổi kích thước.
- Màn hình ô tô hoặc thiết bị XR.

Drive UI from data models

- Giao diện phải là kết quả của trạng thái dữ liệu hiện tại.
- Khi State thay đổi, giao diện được tạo lại để phản ánh trạng thái mới.

Vai trò

- UI luôn nhất quán với dữ liệu.
- Không phải cập nhật thủ công từng thành phần giao diện.
- Dễ khôi phục giao diện sau khi xoay màn hình.
- Dễ kiểm thử: chỉ cần cung cấp một trạng thái và kiểm tra kết quả hiển thị.

Single source of truth

- Mỗi loại dữ liệu phải có một thành phần duy nhất chịu trách nhiệm sở hữu và thay đổi dữ liệu đó.
- Thành phần này được gọi là **Single Source of Truth – SSOT**.
- Các thành phần khác:
  - Chỉ quan sát dữ liệu.
  - Không giữ thêm một bản dữ liệu độc lập.
  - Muốn thay đổi dữ liệu phải gửi yêu cầu đến SSOT.

Unidirectional data flow

Trạng thái và sự kiện di chuyển theo các hướng xác định:

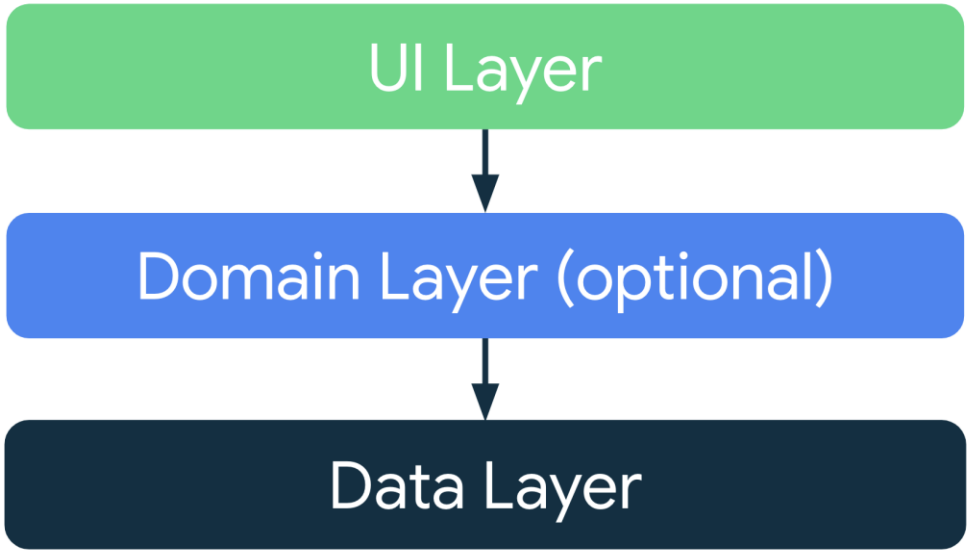
- **State đi từ trên xuống:** nguồn dữ liệu → ViewModel → UI.
- **Event đi từ dưới lên:** UI → ViewModel → nguồn dữ liệu.

Vai trò

- Luồng xử lý rõ ràng và dễ theo dõi.
- Hạn chế việc UI tự ý thay đổi dữ liệu.
- Tránh vòng lặp cập nhật khó kiểm soát.
- Dễ ghi log và tìm lỗi.
- Cho phép UI chỉ phụ thuộc vào trạng thái đầu vào.

Recommended architecture

UI layer - Tầng giao diện: Hiển thị dữ liệu và tiếp nhận thao tác của người dùng.
Data layer - Tầng dữ liệu: Cung cấp, lưu trữ và thay đổi dữ liệu của ứng dụng.
Domain layer - Tầng nghiệp vụ: Tùy chọn, dùng để đơn giản hóa và tái sử dụng tương tác giữa UI và Data.



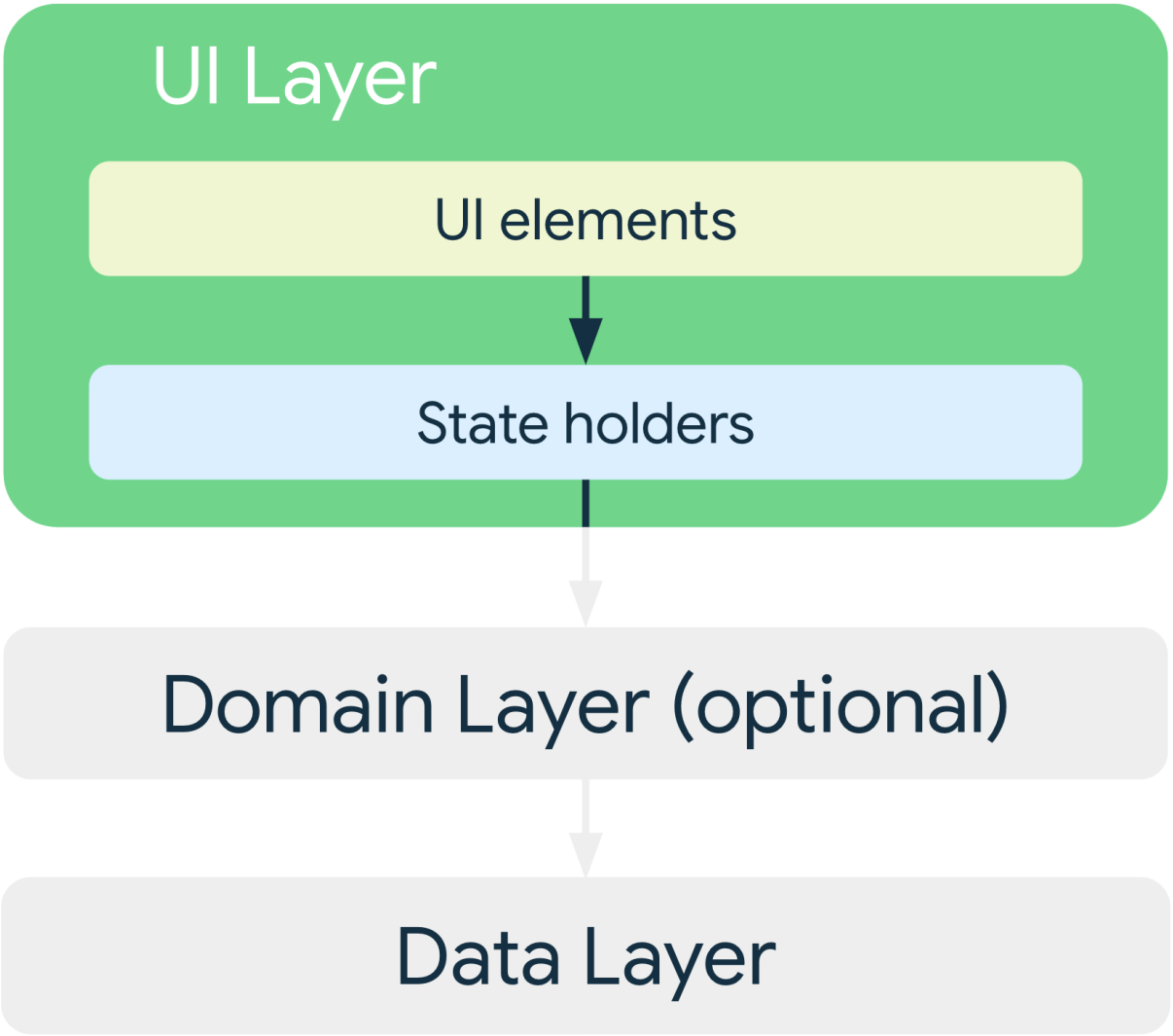
HUST

hust.edu.vn fb.com/dhbkhn

PHẦN 1 – Architecture components

- Vòng đời của activity
- Kiến trúc ứng dụng
- **Lớp giao diện**
- ViewModel và State

Lớp giao diện (UI Layer)



Lớp giao diện

- Vai trò: Hiển thị dữ liệu ứng dụng trên màn hình.
- Khi dữ liệu thay đổi, do
  - Tương tác người dùng
  - Nguồn dữ liệu ngoài

$\Rightarrow$ Giao diện cập nhật để phản ánh sự thay đổi.

- Gồm 2 thành phần:
  - Đối tượng giao diện (UI elements) : hiển thị dữ liệu trên màn hình, sử dụng các đối tượng Jetpack Compose
  - Đối tượng giữ trạng thái (State holders):
    - Chứa dữ liệu của màn hình
    - Cung cấp dữ liệu cho thành phần giao diện
    - Thực hiện nghiệp vụ thay đổi dữ liệu

UI layer gồm state holder và UI elements

STATE HOLDERS

- ViewModel
- Chuẩn bị và cập nhật UI state
- Nhận sự kiện

UI ELEMENTS

- Composable
- Đọc state và vẽ UI
- Phát callback

Unidirectional Data Flow tạo vòng lặp rõ ràng

STATE ↓

- ViewModel phát UiState
- Composable quan sát
- UI hiển thị dữ liệu

EVENT ↑

- Người dùng thao tác
- UI gọi event handler
- ViewModel cập nhật state

HUST

hust.edu.vn fb.com/dhbkhn

PHẦN 1 – Architecture components

- Vòng đời của activity
- Kiến trúc ứng dụng
- Lớp giao diện
- **ViewModel và State**

ViewModel

- ViewModel là lớp dùng để **quản lý dữ liệu và trạng thái của màn hình**.
- ViewModel nhận các sự kiện từ giao diện và xử lý logic cần thiết.
- Khi dữ liệu thay đổi, ViewModel cung cấp trạng thái mới để giao diện cập nhật.
- Dữ liệu trong ViewModel vẫn được giữ khi xoay màn hình hoặc thay đổi cấu hình.
- ViewModel giúp tách phần xử lý dữ liệu khỏi Activity, Fragment và Composable.

Ví dụ khai báo ViewModel

Khai báo thư viện

```
dependencies {
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.11.0")
}
```

Khai báo lớp ViewModel

```
class CounterViewModel : ViewModel() {
    var count by mutableStateOf(0)
        private set

    fun increase() {
        count++
    }
}
```

Ví dụ sử dụng ViewModel

```
@Composable
fun CounterScreen(
    viewModel: CounterViewModel = viewModel()
) {
    Column {
        Text("Giá trị: ${viewModel.count}")

        Button(onClick = viewModel::increase) {
            Text("Tăng")
        }
    }
}
```

StateFlow

- Là luồng dữ liệu luôn chứa **một giá trị trạng thái hiện tại**.
- Khi giá trị thay đổi, các thành phần đang quan sát sẽ nhận được giá trị mới.
- Phù hợp với ViewModel, dữ liệu từ Room, API hoặc nhiều nguồn kết hợp.
- Trong Compose, thường quan sát bằng collectAsState() hoặc collectAsStateWithLifecycle().

So sánh mutableStateOf() và StateFlow

mutableStateOf()

- Gắn trực tiếp với Compose
- Phù hợp trạng thái UI đơn giản
- Compose tự quan sát
- Khó kết hợp nhiều nguồn

StateFlow

- Không phụ thuộc vào Compose
- Phù hợp dữ liệu bất đồng bộ
- Cần collectAsState()
- Linh hoạt khi kết hợp nhiều nguồn

Ví dụ

Khai báo StateFlow trong ViewModel

```
private val _count = MutableStateFlow(0)
val count = _count.asStateFlow()
fun increase() {
    _count.value++
}
```

Quan sát StateFlow trong Composable

```
val count by viewModel.count.collectAsState()
Column {
    Text("Giá trị: $count")
    Button(onClick = viewModel::increase) { Text("Tăng") }
}
```

Các hàm biến đổi dữ liệu

map	Biến đổi mỗi giá trị thành một giá trị mới
filter	Chỉ giữ lại các giá trị thỏa mãn điều kiện
combine	Kết hợp giá trị mới nhất của nhiều Flow

UiState mô tả toàn bộ màn hình

UiState là một đối tượng chứa **toàn bộ trạng thái cần thiết để hiển thị một màn hình** tại một thời điểm.

Vai trò của UiState

- Tập trung trạng thái của màn hình vào một đối tượng.
- Giúp UI luôn hiển thị đúng theo dữ liệu hiện tại.
- Tránh khai báo quá nhiều biến trạng thái rời rạc.
- Dễ cập nhật, kiểm thử và tìm lỗi.
- Phù hợp với nguyên lý **UI được điều khiển bởi dữ liệu**.

Khi UiState thay đổi, Compose tự cập nhật giao diện.

Ví dụ

Màn hình danh sách công việc cần biết:

- Danh sách công việc hiện tại.
- Có đang tải dữ liệu không.
- Có thông báo lỗi không.
- Nội dung người dùng đang nhập.

```
data class TodoUiState(
    val todos: List<Todo> = emptyList(),
    val newTitle: String = "",
    val isLoading: Boolean = false,
    val errorMessage: String? = null
)
```

Ví dụ - Sử dụng UiState trong ViewModel

```
class TodoViewModel : ViewModel() {

    private val _uiState = MutableStateFlow(TodoUiState())
    val uiState = _uiState.asStateFlow()

    fun updateTitle(title: String) {
        _uiState.update {
            it.copy(newTitle = title)
        }
    }
}
```

Cập nhật state bằng copy()

- Đọc state hiện tại
- Tạo bản sao với trường thay đổi
- Gán lại atomically qua update {}
- Giữ data class bất biến

Ví dụ - Sử dụng trong Compose

```
@Composable
fun TodoScreen(viewModel: TodoViewModel = viewModel()) {
    val uiState by viewModel.uiState.collectAsState ()

    TextField(
        value = uiState.newTitle,
        onValueChange = viewModel::updateTitle
    )

    if (uiState.isLoading) {
        CircularProgressIndicator()
    }
}
```

Codelab

- https://developer.android.com/codelabs/basic-android-kotlin-compose-viewmodel-and-state#0
- https://developer.android.com/codelabs/basic-android-kotlin-compose-practice-viewmodel

HUST

hust.edu.vn fb.com/dhbkhn

PHẦN 2

NAVIGATION IN COMPOSE

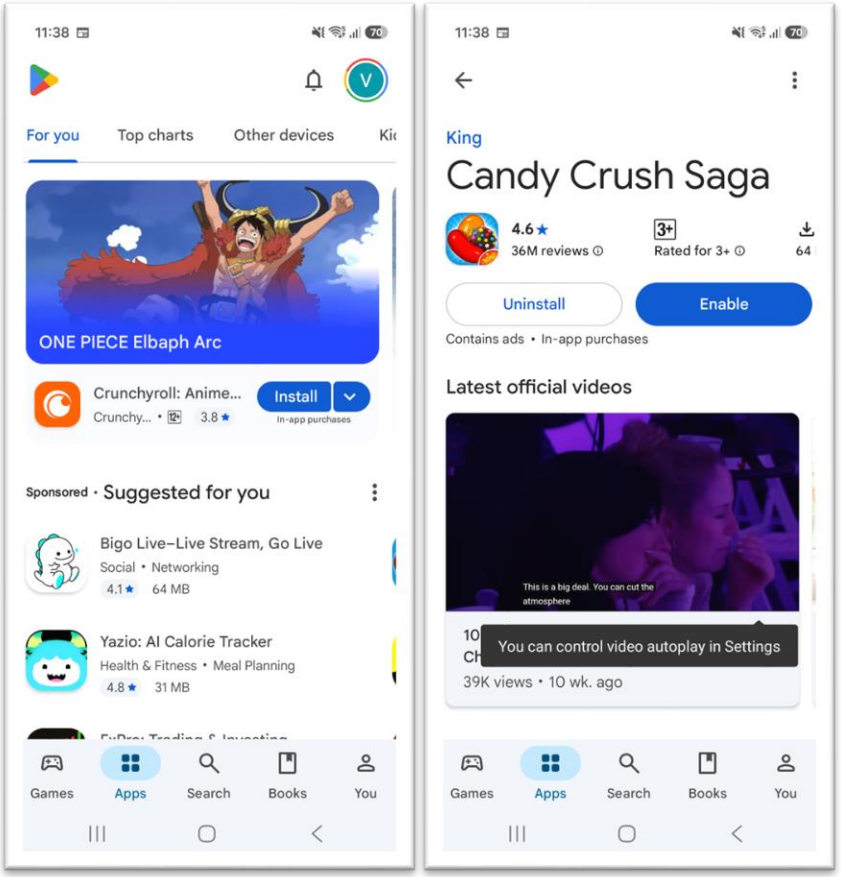
- Routes
- NavHost
- NavController
- Back stack

Giới thiệu

Các ứng dụng thường có nhiều màn hình. Mỗi màn hình thực hiện một chức năng nhất định.

Với Jetpack Compose:

- Một activity
- Mỗi màn hình là một hàm Composable



Giới thiệu

Các đối tượng navigation giúp lập trình viên xử lý nhất quán các thao tác khi di chuyển màn hình:

- Điều hướng giữa các màn hình trong ứng dụng
- Quản lý lịch sử các màn hình đã mở, cho phép quay lại màn hình trước đó
- Truyền dữ liệu giữa các màn hình
- Xử lý các thao tác Back và Up
  - **Back**: quay lại màn hình trước đó trong lịch sử di chuyển
  - **Up**: đi lên màn hình cha trong cấu trúc ứng dụng

Giới thiệu

Các tính năng khác mà thư viện hỗ trợ

- Quản lý Back Stack tự động
- Truyền tham số an toàn về kiểu
- Hỗ trợ ViewModel, chia sẻ dữ liệu giữa các màn hình
- Hỗ trợ Deep Link, mở các màn hình sâu bên trong
- Hỗ trợ hiệu ứng di chuyển màn hình
- Hỗ trợ các thiết kế Bottom Navigation và Navigation Drawer.

Các đối tượng của thư viện Navigation Compose

Đối tượng	Vai trò
NavController	Điều khiển di chuyển giữa các màn hình trong ứng dụng.
NavGraph	Mô tả màn hình và các quan hệ điều hướng.
NavHost	Đối tượng hiển thị màn hình hiện tại.

Các khái niệm bổ sung:

- **Route**: Định danh một màn hình
- **Destination**: Màn hình đích, là một hàm Composable
- **NavBackStackEntry**: một phần tử trong Back Stack, chứa tham số truyền vào màn hình.

Khai báo thư viện

```
dependencies {
    // Thư viện Navigation Compose
    val navVersion = "2.9.8“
    implementation("androidx.navigation:navigation-compose:$navVersion")
}
```

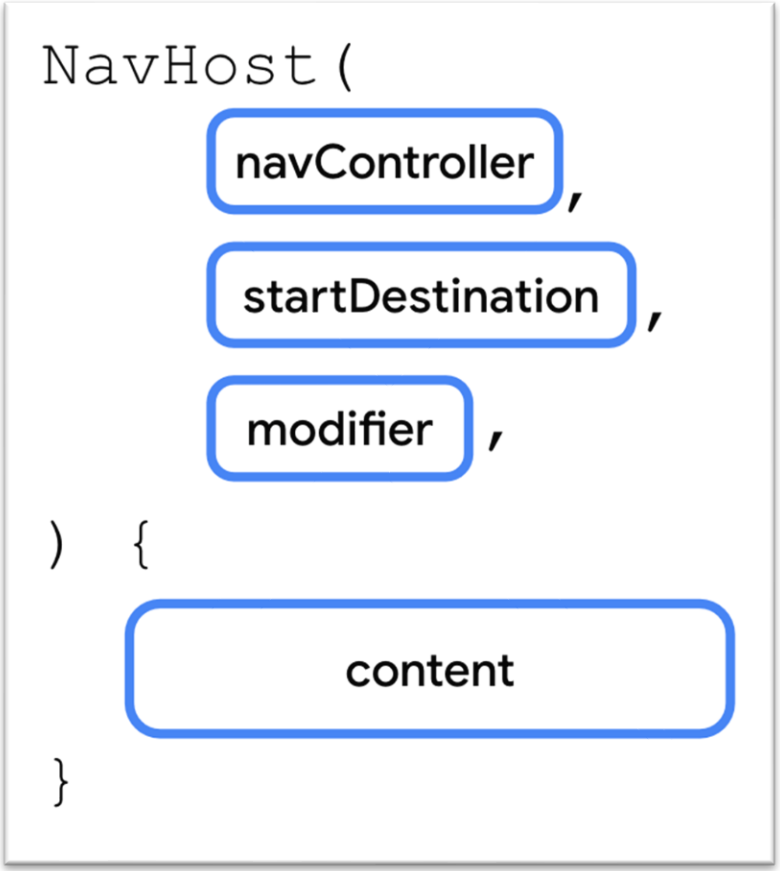
Định nghĩa route đến các màn hình

- Route là định danh của các màn hình
  - Tương tự URL trong ứng dụng web
  - Mỗi màn hình có một route rõ ràng
  - Tên route ổn định và dễ đọc
- Sử dụng **enum** để khai báo tập route cố định
  - Dùng name làm route đơn giản
  - Tránh lỗi chính tả chuỗi

```
enum class CupcakeScreen {
    Start,
    Flavor,
    Pickup,
    Summary
}
```

Thêm NavHost vào màn hình chính

- Là hàm Composable để hiển thị các hàm khác tùy thuộc vào route hiện tại.
- **navController**: đối tượng để di chuyển màn hình, có thể khởi tạo bằng hàm **rememberNavController()**
- **startDestination:** route đến màn hình hiển thị đầu tiên
- **content:** gắn route với các hàm Composable

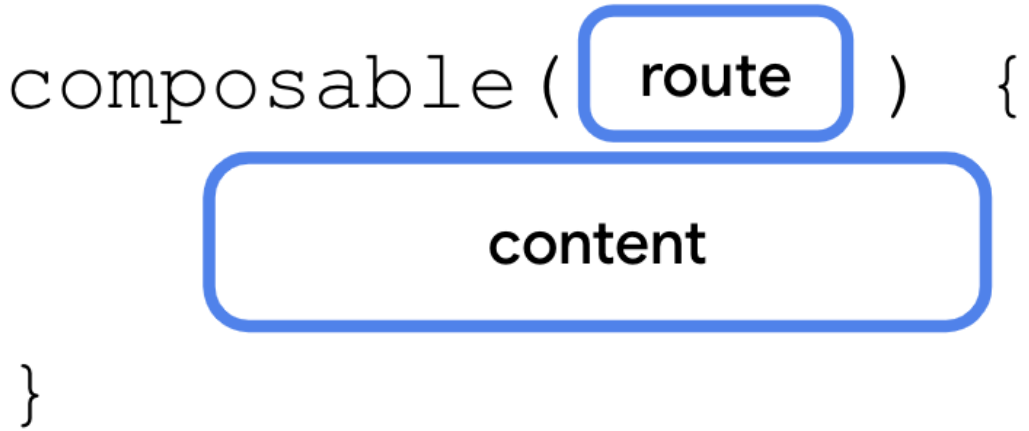


Xử lý các route trong NavHost

Trong nội dung của **NavHost**, gọi các hàm **composable()**

Hàm gồm 2 tham số:

- **route**: Chuỗi ký tự định danh route. Sử dụng thuộc tính **name** để sử dụng các hằng trong **enum**.
- **content**: Gọi hàm **Composable** để hiển thi màn hình tương ứng với



Mỗi composable() ánh xạ một route

- route xác định destination
- Lambda gọi screen composable
- Truyền state và callback cần thiết
- NavHost giữ cấu trúc luồng tập trung

Ví dụ

```
val navController = rememberNavController()
NavHost(navController, CupcakeScreen.Start.name,
    modifier = Modifier.padding(innerPadding)) {
    composable(CupcakeScreen.Start.name) {
    }
    composable(CupcakeScreen.Flavor.name) {
    }
    composable(CupcakeScreen.Pickup.name) {
    }
    composable(CupcakeScreen.Summary.name) {
    }
}
```

Khai báo callback xử lý điều hướng ở các màn hình

- Phương án: truyền navController vào các màn hình để thực hiện điều hướng => Hoạt động nhưng không tối ưu
- Nên giữ việc điều hướng trong phạm vi của NavHost, tách biệt với giao diện
  - Thuận tiện trong việc bảo trì
  - Phù hợp với ứng dụng chạy trên nhiều dạng màn hình
- Cách thực hiện:
  - Sử dụng các kiểu hàm (lambda, callback) để truyền vào các hàm màn hình.
  - Khi các sự kiện xảy ra, hàm callback tương ứng được gọi.

Khai báo callback xử lý điều hướng ở các màn hình

NÊN

- onNext()
- onCancel()
- Screen độc lập và dễ test

TRÁNH

- Screen gọi navController trực tiếp
- Logic điều hướng bị phân tán
- Khó thích ứng nhiều form factor

Ví dụ

```
fun StartOrderScreen(
    quantityOptions: List<Pair<Int, Int>>,
    onNextButtonClicked: () -> Unit,
    modifier: Modifier = Modifier
) {
...
}
```

- Hàm callback được truyền vào màn hình.
- Được gọi khi cần di chuyển màn hình khác.
- Nội dung hàm được định nghĩa trong phạm vi NavHost

Thực hiện điều hướng

- Sử dụng hàm navigate của đối tượng NavController
- Cập nhật state trước khi chuyển màn hình
- Gọi navController.navigate(route)

```
composable(CupcakeScreen.Start.name) {
    StartOrderScreen(
        DataSource.quantityOptions,
        onNextButtonClicked = {
            viewModel.setQuantity(it)
            navController.navigate(CupcakeScreen.Flavor.name)
        },
        Modifier
            .fillMaxSize()
            .padding(dimensionResource(R.dimen.padding_medium))
    )
}
```

Truyền tham số đến màn hình sau

```
composable(
    route = "detail/{productId}?name={name}",
    arguments = listOf(
        navArgument("productId") {
            type = NavType.IntType
        },
        navArgument("name") {
            type = NavType.StringType
            defaultValue = ""
        }
    )
) { backStackEntry ->
    val productId =
        backStackEntry.arguments?.getInt("productId") ?: 0
    val productName =
        backStackEntry.arguments?.getString("name").orEmpty()
    ...
}
```

```
navController.navigate("detail/$id?name=${Uri.encode(name)}")
```

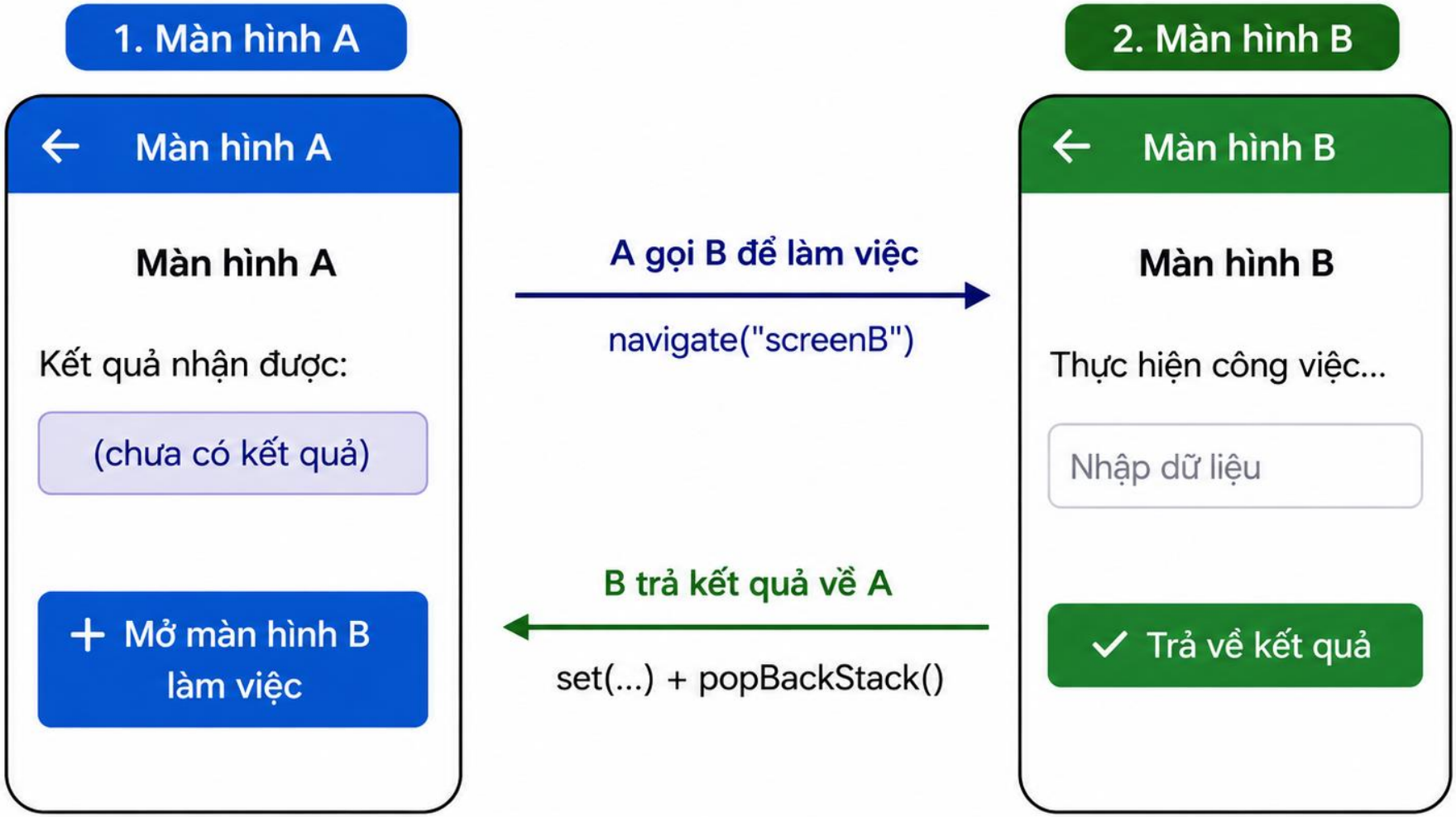
Định nghĩa cấu trúc route

Định nghĩa kiểu dữ liệu và giá trị mặc đinh

Lấy tham số tại màn hình đích

Truyền tham số khi chuyển màn hình

Trả về kết quả cho màn hình trước



Trả về kết quả cho màn hình trước – Ví dụ

```
composable("home") { backStackEntry ->
    val selectedName by backStackEntry
        .savedStateHandle
        .getStateFlow<String?>(
            key = "selected_name",
            initialValue = null
        )
        .collectAsStateWithLifecycle()

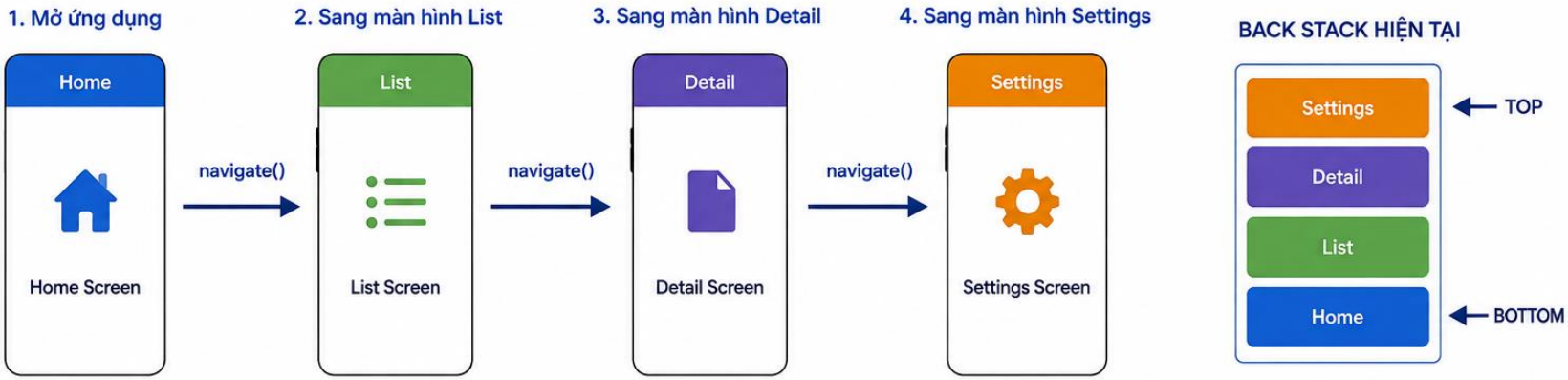
    HomeScreen(
        selectedName = selectedName,
        onOpenSelection = {
            navController.navigate("selection")
        },
        onResultHandled = {
            backStackEntry.savedStateHandle
                .remove<String>("selected_name")
        }
    )
}
```

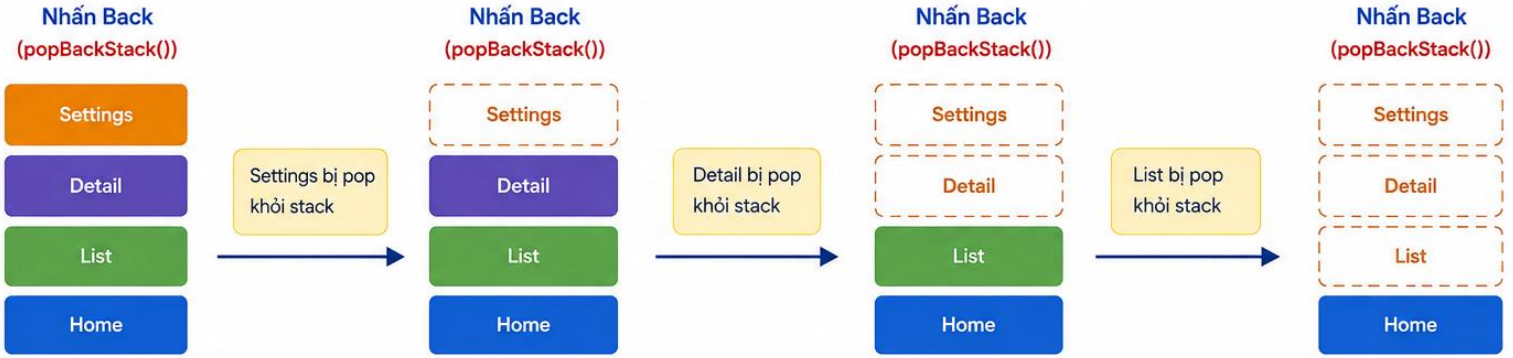
```
composable("selection") {
    SelectionScreen(
        onSelect = { name ->

            // Gửi kết quả về màn hình trước
            navController.previousBackStackEntry
                ?.savedStateHandle
                ?.set("selected_name", name)

            // Quay lại màn hình trước
            navController.popBackStack()
        }
    )
}
```

Back Stack lưu lại lịch sử di chuyển màn hình





- Mỗi khi navigate() -> màn hình mới được đẩy lên đỉnh (push).
- Khi nhấn Back hoặc gọi popBackStack() -> màn hình ở đỉnh bị pop ra khỏi stack, màn hình bên dưới sẽ hiện ra.
- Nếu stack chỉ còn 1 màn hình (Home), nhấn Back sẽ thoát ứng dụng.

navigate()	= push màn hình lên stack
popBackStack()	= pop màn hình khỏi stack (Back)

popBackStack xử lý nút Back tùy chỉnh

popBackStack() => quay trở lại màn hình liền trước trong backstack

popBackStack(

route: String, inclusive: Boolean, saveState: Boolean

)

=> quay trở lại màn hình bất kỳ trong backstack

Các tham số:

route: màn hình cần quay lại, tồn tại trong backstack

inclusive: có xóa màn hình quay lại khỏi ngăn xếp không

saveState: có lưu trạng thái của màn hình quay lại không

Codelab

https://developer.android.com/codelabs/basic-android-kotlin-compose-navigation#0

https://developer.android.com/codelabs/basic-android-kotlin-compose-practice-navigation

HUST

hust.edu.vn fb.com/dhbkhn

PHẦN 2

Di chuyển đến màn hình của ứng dụng khác

Intent kết nối ứng dụng với hệ thống

Đối tượng thông điệp mô tả hành động mà ứng dụng muốn thực hiện.

- Có thể dùng để mở Activity, khởi động Service hoặc gửi Broadcast.
- Dùng Intent để yêu cầu một Activity xử lý gọi điện, soạn SMS, mở trang web hoặc bản đồ.
- Intent mô tả yêu cầu; hệ thống Android tìm thành phần phù hợp để thực hiện yêu cầu đó.

Hai loại Intent

EXPLICIT INTENT

- Chỉ rõ chính xác Activity hoặc component đích.
- Thường dùng để chuyển màn hình bên trong cùng ứng dụng.

=> Ngoài phạm vi khóa học này.

IMPLICIT INTENT

- Mô tả hành động và dữ liệu, không chỉ rõ ứng dụng đích.
- Android chọn ứng dụng có intent filter phù hợp.

Implicit Intent hoạt động như thế nào?

1. **Tạo Intent** — khai báo action, data và extras nếu cần.
2. **Gửi yêu cầu** — gọi context.startActivity(intent).
3. **Android đối sánh** — so sánh Intent với intent filter của các ứng dụng đã cài.
4. **Mở đích** — một ứng dụng phù hợp được mở; nếu có nhiều lựa chọn, Android có thể hiển thị hộp chọn.

Các thành phần quan trọng của Intent

Action: hành động cần thực hiện, ví dụ ACTION_VIEW, ACTION_DIAL, ACTION_SENDTO.

Data: URI mô tả dữ liệu: tel:, smsto:, https:, geo:.

Category: thông tin bổ sung về loại component được phép xử lý.

Extras: dữ liệu dạng key-value, ví dụ nội dung tin nhắn.

Flags: điều chỉnh cách Android khởi chạy hoặc quản lý Activity.

Ví dụ: Mở ứng dụng gọi điện

```
val context = LocalContext.current
Button(onClick = {
    val intent = Intent(Intent.ACTION_DIAL, "tel:0987654321".toUri())
    context.startActivity(intent)
}) {
    Text("Open phone")
}
```

Ví dụ: Soạn tin nhắn

```
val context = LocalContext.current
Button(onClick = {
    val intent = Intent(Intent.ACTION_SENDTO, "smsto:0987654321".toUri())
    intent.putExtra("sms_body", "Hello!")
    context.startActivity(intent)
}) {
    Text("Send SMS")
}
```

Ví dụ: Mở trang web

```
val context = LocalContext.current
Button(onClick = {
    val intent = Intent(Intent.ACTION_VIEW, "https://hust.edu.vn".toUri())
    context.startActivity(intent)
}) {
    Text("Open webpage")
}
```

Ví dụ: Mở bản đồ

```
val context = LocalContext.current
Button(onClick = {
    val place = Uri.encode(
        "Hanoi University of Science and Technology"
    )
    val geoUri = "geo:0,0?q=$place".toUri()
    val intent = Intent(Intent.ACTION_VIEW, geoUri)
    context.startActivity(intent)
}) {
    Text("Open Map")
}
```

PHẦN 3

Giao diện thích ứng

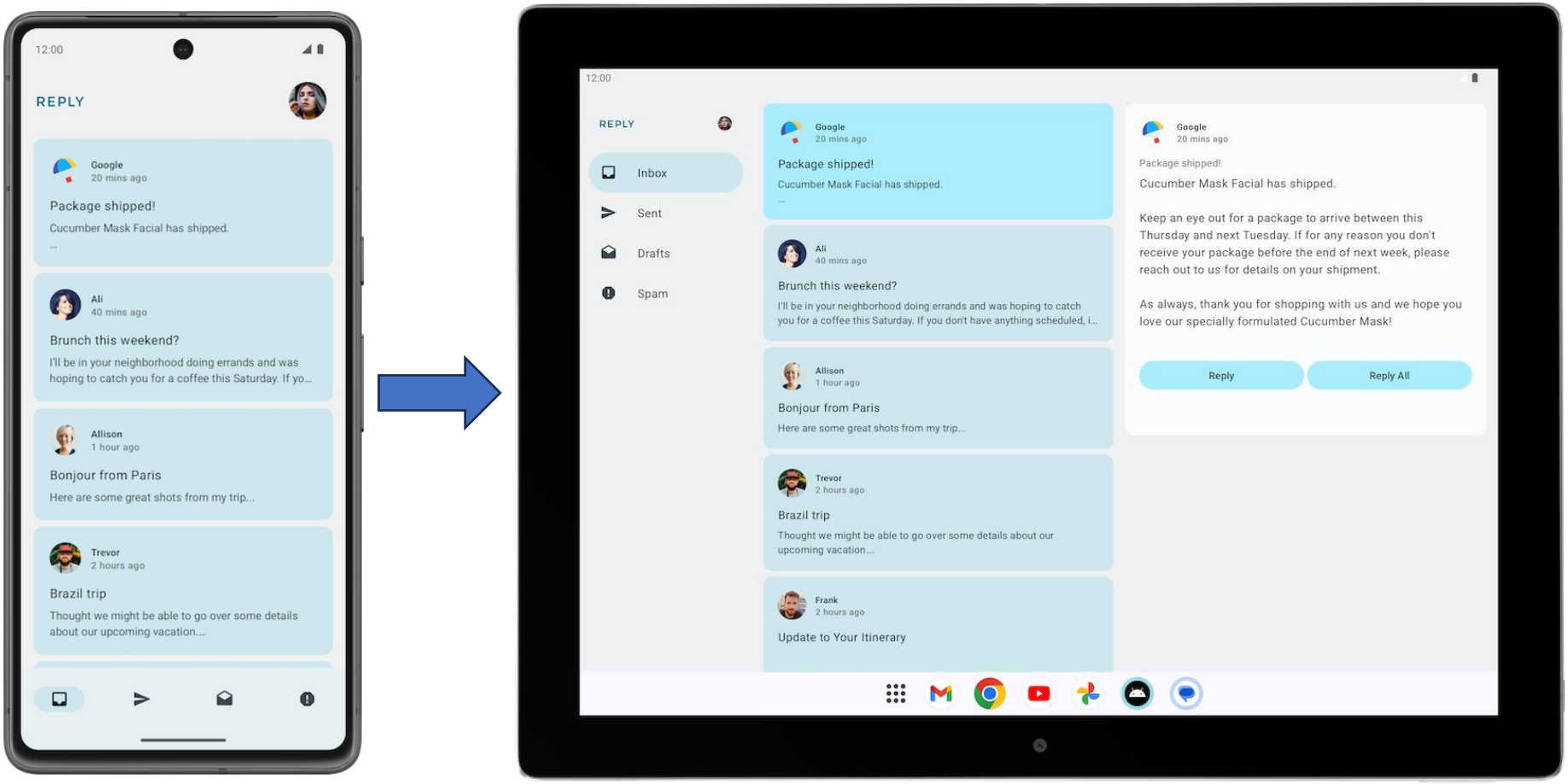
- Window size classes
- Dynamic navigation
- List–detail

Hệ sinh thái Android đa dạng

- **Điện thoại:** màn hình nhỏ, cảm ứng, thao tác một tay
- **Thiết bị gập & máy tính bảng**: màn hình linh hoạt, đa nhiệm
- **Đồng hồ, TV & ô tô**: cách nhập liệu và khoảng cách tương tác khác nhau
- Giao diện phải thích ứng theo kích thước và ngữ cảnh sử dụng



Vấn đề đặt ra khi triển khai trên nhiều kiểu thiết bị



Adaptive UI cho nhiều màn hình

- Điện thoại, tablet, foldable, desktop
- Kích thước cửa sổ có thể thay đổi
- Tận dụng không gian thay vì kéo giãn
- Giữ trải nghiệm nhất quán

Responsive và adaptive bổ trợ nhau

RESPONSIVE

- Co giãn thành phần
- Reflow theo không gian
- Giữ cấu trúc

ADAPTIVE

- Đổi cấu trúc/pattern
- Thêm pane/navigation
- Tối ưu theo size class

Không suy ra layout chỉ từ loại thiết bị

THIẾT BỊ

- Tablet có thể chạy split-screen
- Foldable thay đổi tư thế

CỬA SỔ

- Đo không gian ứng dụng hiện có
- Ra quyết định theo width/height
- Phản ứng khi resize

Breakpoint là gì?

- Mục tiêu: Thay đổi giao diện tùy thuộc vào kích thước màn hình.
- Breakpoint là các điểm mốc về kích thước.

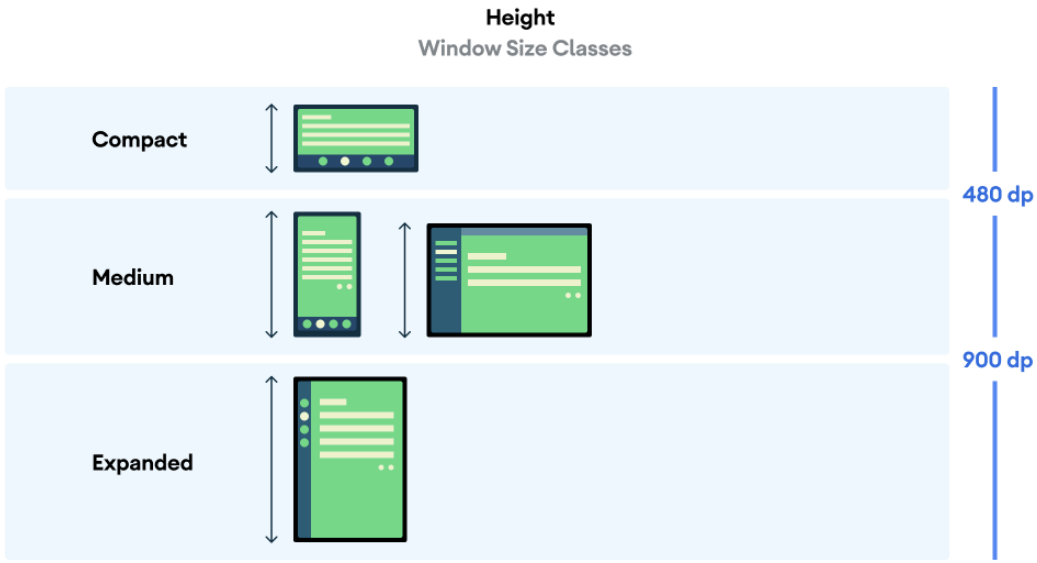
Breakpoint range (dp)	Portrait	Landscape	Window size class	Columns	Minimum margins*
0-599	Handset	Phone**	Compact	4	8
600-839	Foldable Small tablet	Foldable Small tablet	Medium	12	12
840+	Large tablet	Large tablet	Expanded	12	32

*Margins and gutters are flexible and don't need to be equal in size.
**Phones in landscape are considered an exception in order to still fit within this category.

Sử dụng các lớp Window Size

- Phân loại chiều rộng và chiều cao
  - Compact
  - Medium
  - Expanded
- Dùng breakpoint thay vì model thiết bị
- Cập nhật khi kích thước cửa sổ đổi





Compact ưu tiên một nhiệm vụ mỗi lần

ĐẶC TRƯNG

- Chiều rộng hạn chế
- Thường là điện thoại dọc

THIẾT KẾ

- Một pane nội dung
- Bottom navigation
- Điều hướng tới detail

Medium tăng mật độ hiển thị

ĐẶC TRƯNG

- Không gian trung gian
- Điện thoại ngang
- Tablet nhỏ

THIẾT KẾ

- Navigation rail
- Pane rộng hơn
- Cân nhắc list–detail

Expanded tận dụng nhiều pane

ĐẶC TRƯNG

- Không gian ngang lớn
- Tablet/desktop window rộng

THIẾT KẾ

- Drawer hoặc rail
- List và detail đồng thời
- Giảm chuyển màn hình

Sử dụng lớp Windows Size

Khai báo thư viện

```
implementation("androidx.compose.material3:material3-
  window-size-class")
```

Tính toán kích thước

```
val windowSize = calculateWindowSizeClass(this)
val windowWidthSize = windowSize.widthSizeClass
```

Tính kích thước tại app level

- Quan sát WindowSizeClass hiện tại
- Chuyển class thành kiểu navigation
- Truyền quyết định xuống layout

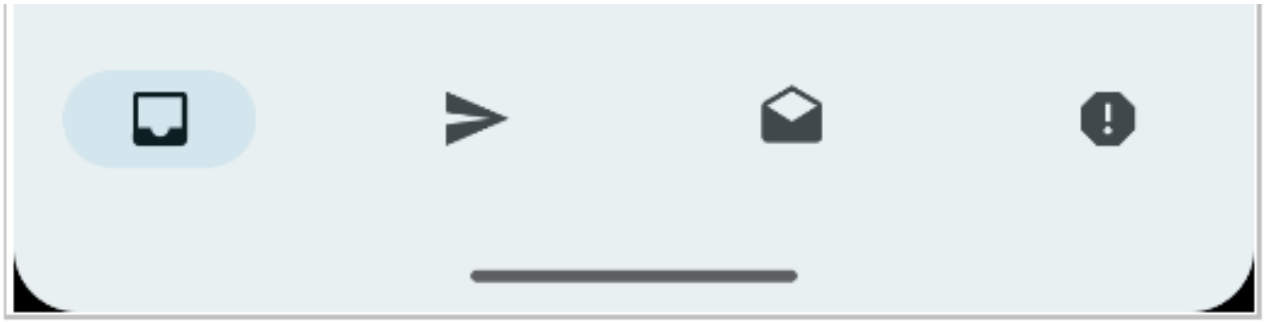
```
@Composable
fun ReplyApp( windowSize: WindowWidthSizeClass) {
    ...
    when (windowSize) {
        WindowWidthSizeClass.Compact -> {
        }
        WindowWidthSizeClass.Medium -> {
        }
        WindowWidthSizeClass.Expanded -> {
        }
        else -> {
        }
    }
...
```

Navigation thích ứng theo chiều rộng

- Compact: NavigationBar phía dưới
- Medium: NavigationRail bên cạnh
- Expanded: Permanent drawer khi phù hợp

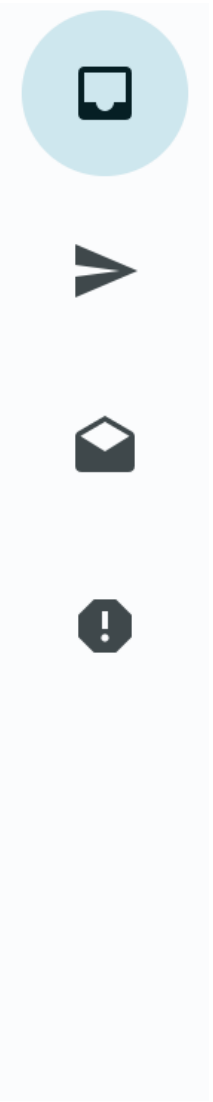
Bottom bar phù hợp màn hình hẹp

- Nằm ở cạnh dưới dễ chạm
- Hiển thị số destination chính vừa phải
- Giữ label và icon rõ ràng
- Không chiếm chiều ngang nội dung



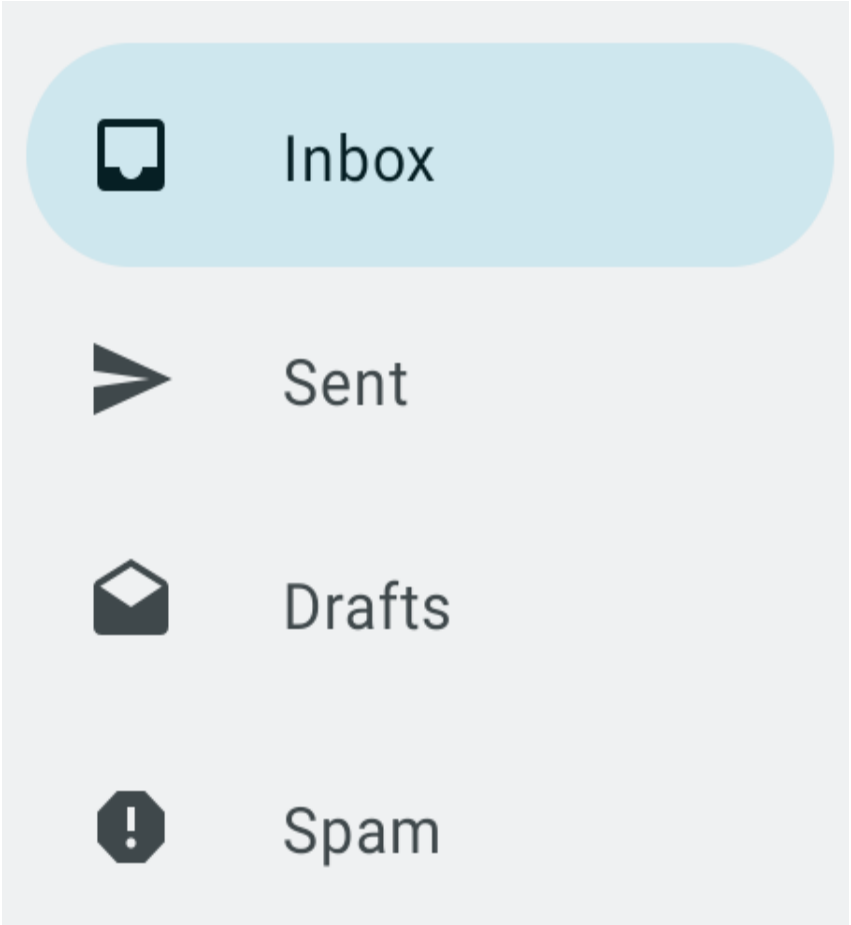
Navigation rail tận dụng cạnh dọc

- Phù hợp medium width
- Giải phóng chiều cao cho nội dung
- Có thể compact hoặc expanded
- Giữ cùng destination model



Drawer phù hợp không gian rộng

- Hiển thị navigation thường trực
- Có chỗ cho label rõ hơn
- Không che nội dung khi permanent
- Chọn theo nhu cầu chứ không máy móc



Codelab

https://developer.android.com/codelabs/basic-android-kotlin-compose-adaptive-navigation-for-large-screens

https://developer.android.com/codelabs/basic-android-kotlin-compose-adaptive-content-for-large-screens

https://developer.android.com/codelabs/basic-android-kotlin-compose-my-city

HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.